COSC 6385 Assignment 1:
Branch Prediction and Multi-Issue Tomasulo

Aymane Khaldi

October 21, 2026

Code repository: `https://github.com/Aymane-Khaldi03/assignment-1-ComputerArchi`

Part 1 ("`branch-prediction/`") implements a Pentium M-style branch direction predictor in the CBP2 infrastructure. Part 2 ("`o3-simulator/`") extends the provided Python Tomasulo simulator to issue 1 to 4 instructions per cycle in order.

1 Part 1: Pentium M-Style Hybrid Branch Direction Predictor

1.1 Summary

The `pm_predictor` class in "`src/my_predictor.h`" of the CBP2 infrastructure models the branch outcome part of the Pentium M predictor: a bimodal table plus a tagged, set-associative global predictor. As the assignment allows, the loop predictor, the PIR and its hash, and branch target prediction are not implemented. With the handout's toy parameters, the predictor reproduces all 22 steps of the hybrid predictor example in the handout. On the 20 CBP2 traces the default configuration averages **5.849 MPKI**, compared with 10.267 for a bimodal table alone and 6.306 for the 32K-entry gshare predictor supplied with the infrastructure.

1.2 Design

Bimodal predictor. $2^{12} = 4096$ two-bit saturating counters indexed by the low 12 bits of the branch address (IP[11:0]). A counter value ≥ 2 predicts taken. The bimodal counter is trained on every conditional branch.

Global predictor. A cache of two-bit counters with 512 sets, 4 ways (2048 entries), 6-bit tags, a valid bit and true LRU replacement. The global history register (GHR) holds the last 15 conditional branch outcomes. The predictor computes

$$h = (\text{address} \oplus \text{GHR}) \bmod 2^{15}, \qquad \text{set} = h[14{:}6], \qquad \text{tag} = h[5{:}0],$$

which gives the same index and tag fields as the Pentium M diagram in the handout: the upper bits select the set and the lower bits form the tag, as in the handout example. The sizes follow the Pentium M organization reported by Uzelac and Milenkovic [1]. The GHR replaces the PIR and its hash, which the handout does not require.

Selection. If the tag hits in the selected set, the global counter gives the prediction. Otherwise the bimodal prediction is used.

Update. On a global hit, the hit entry's counter is trained and the entry becomes MRU. On a global miss, an entry may be allocated in the set's victim way: an invalid way if one exists, otherwise the LRU way. Two allocation policies were evaluated:

- *Allocate on every miss, keep the old counter.* This is the policy of the handout example: the victim's counter is reused and trained with the outcome.
- *Allocate only when the bimodal prediction was wrong, and initialize the counter weakly toward the outcome* (2 if taken, 1 if not). New entries are created only for branches that the bimodal table has mispredicted. This is the default.

Initialization and unconditional branches. Unconditional branches are predicted taken and do not change any state. The target prediction is always 0. All counters start weakly taken (2), global entries start invalid, and the GHR starts at zero.

All parameters are compile-time macros, so one class covers the handout's toy predictor, the full-size predictor, and the following variants: `PM_BIMODAL_BITS`, `PM_SET_BITS`, `PM_WAYS`, `PM_TAG_BITS`, `PM_ADDR_SHIFT`, `PM_GLOBAL`, `PM_ALLOC_ON_MISPREDICT` and `PM_ALLOC_RESET_COUNTER`. No external cache implementation was imported. Set-associative lookup and LRU replacement are implemented directly in this class.

1.3 Verification Against the Handout Example

"`tests/test_example.cc`" builds the toy configuration from Appendix B of the handout: 8-bit addresses indexed by bits [5:2], 16 bimodal counters, 4 sets × 2 ways with 2-bit tags, and a 4-bit GHR. It loads the handout's initial counter values and the allocate-on-every-miss policy, then replays the branch sequence of b1, b2 and b3. For every step it checks the GHR, the row and tag, the local prediction, the global hit or miss, and the final prediction against the handout tables. All 22 worked steps match, with 9 mispredictions, the same as the handout. (The branch sequence on page 7 lists a 23rd branch, but the worked tables end at step 22.)

1.4 Methodology

Each configuration ran on the 20 traces distributed with the infrastructure [2] (SPEC CPU2000 integer and SPECjvm98, 100M instructions each). The metric, as reported by `predict`, is direction mispredictions of conditional branches per 1000 instructions (MPKI). The default predictor was measured with the provided `run traces` script. "`experiments.sh`" builds the other configurations with `-D` flags and runs them on the same traces. All binaries were built with `g++ -O3 -Wall` (GCC 13.3, Ubuntu 24.04 under WSL). Each trace starts with a fresh predictor, and averages are arithmetic means over the 20 traces. The gshare baseline keeps its supplied initialization (all counters strongly not taken).

1.5 Results

"Table 1" gives per-trace MPKI for the two baselines and both allocation policies of the 4-way predictor. "Table 2" compares the averages and storage of all configurations. Storage counts 2 bits per counter, plus 1 valid bit, 6 tag bits and $\log_2(\text{ways})$ LRU bits per global entry, plus the GHR (1 KiB = 1024 bytes). For the default this is $4096 \cdot 2 + 2048 \cdot (2 + 1 + 6 + 2) + 15 = 30{,}735$ bits, or 3.75 KiB. This is logical hardware storage, not the memory used by the C++ simulator.

Table 1: MPKI per trace (lower is better). PM = Pentium M-style hybrid, 4-way, 512 sets. "Every miss" allocates on every global miss (handout policy); "selective" allocates only when the bimodal table mispredicts (default).

Trace	Bimodal 4K	gshare 32K	PM every miss	PM selective
164.gzip	15.031	12.473	12.692	12.439
175.vpr	14.673	13.415	15.967	13.298
176.gcc	19.313	11.254	17.999	9.266
181.mcf	33.271	15.837	18.269	15.624
186.crafty	13.097	5.837	9.105	5.208
197.parser	15.420	10.008	10.933	8.542
201.compress	9.368	7.831	8.017	7.688
202.jess	8.773	1.562	2.000	1.124
205.raytrace	3.704	2.756	3.597	2.293
209.db	6.437	3.909	4.824	3.608
213.javac	3.198	2.267	3.008	2.052
222.mpegaudio	3.166	2.188	2.940	2.031
227.mtrt	3.682	2.657	3.436	2.389
228.jack	7.175	3.033	4.238	2.349
252.eon	9.443	1.807	2.148	1.506
253.perlbmk	5.848	2.554	5.084	1.774
254.gap	9.760	3.926	4.333	3.509
255.vortex	1.930	1.222	1.641	0.830
256.bzip2	0.113	0.094	0.112	0.062
300.twolf	21.943	21.489	24.817	21.390
Average	10.267	6.306	7.758	**5.849**

1.6 Discussion

Effect of the global predictor. Adding the global predictor to the bimodal table lowers the average MPKI by 43% (10.267 to 5.849). Large reductions appear on eon (9.4 to 1.5), jess (8.8 to 1.1), gcc (19.3 to 9.3) and mcf (33.3 to 15.6). The default PM predictor is lower than the 8 KiB gshare on all 20 traces, with less than half its storage; on bzip2, gzip, twolf and vpr the two are within 0.12 MPKI of each other. A likely reason is that the tags and selective allocation keep easily predicted branches out of the global table, which reduces interference compared with gshare's untagged table.

Effect of the allocation policy. The handout's allocate-on-every-miss policy averages 7.758 MPKI, higher than gshare (6.306). Every miss allocates or replaces an entry, so the 2048 entries are spread over all (address, history) pairs, including the many branches that the bimodal table already predicts correctly. Useful entries can then be evicted before they are trained; for example, gcc reaches 17.999 MPKI instead of 9.266. With the counter policy held fixed, allocating only after a bimodal misprediction lowers the average MPKI by 23.5% (7.758 to 5.935). Starting a new entry's counter weakly toward the outcome, instead of inheriting the evicted entry's counter, lowers the average by a further 0.09 MPKI.

Associativity and capacity. Going from 4 to 2 ways at the same number of sets halves the global table and raises the average MPKI to 6.134. A 2-way table with 1024 sets has the same 2048 entries as the 4-way default and nearly the same MPKI (5.868 vs. 5.849). That configuration also

Table 2: Average MPKI and storage of each configuration.

Configuration	Allocation policy	Storage (KiB)	Avg. MPKI
Bimodal only, 4K entries	—	1.00	10.267
gshare, 32K entries, 15-bit history	—	8.00	6.306
PM 4-way × 512 sets	every miss, keep counter	3.75	7.758
PM 4-way × 512 sets	on mispredict, keep counter	3.75	5.935
PM 4-way × 512 sets (default)	on mispredict, reset counter	3.75	**5.849**
PM 2-way × 512 sets	on mispredict, reset counter	2.25	6.134
PM 2-way × 1024 sets	on mispredict, reset counter	3.50	5.868

uses a 16-bit instead of a 15-bit history, so the effects of associativity and history length are not fully separated. With the same number of entries, the two-way design permitted by the handout is within 0.02 MPKI of the default.

Limitations. Four traces (twolf, mcf, vpr and gzip) remain above 12 MPKI. The loop predictor and the path-based PIR of the real Pentium M were not modeled, so their effect on these traces was not measured. The XOR/GHR hash, true LRU and the initialization are simulation choices: this is a simplified Pentium M-style predictor, not a bit-exact model of the processor.

1.7 Files

Paths are relative to "`branch-prediction/cbp2-infrastructure-v2/`". The predictor is in "`src/my_predictor.h`". "`tests/`" replays the handout example, `run traces` runs the default predictor, and "`experiments.sh`" builds all variants and writes "`results/results.csv`". "`branch-prediction/README.md`" gives dependencies and commands.

2 Part 2: Tomasulo Algorithm with Multi-Issue

2.1 Summary

The provided Python simulator ("`tomasulo-python`") issues one instruction per cycle. It now issues 1 to 4 instructions per cycle in program order, set with `Issue width = N` in the input file (default 1). The provided code also lacked parts of the processor described in the handout (Table 3); they were added so that the simulator follows the handout. Seven test programs run at every issue width, and all 28 runs end with the same registers and memory as an in-order execution of the program.

2.2 What Was Added to the Code

The file structure of the provided code is kept: "`main.py`" runs the stages, and each module implements one stage. "Table 4" lists each rule of the handout and where it is implemented.

Correction of a renaming error. In the provided code a broadcast also wrote the RAT, so an older instruction that completed later replaced the RAT entry of a younger instruction writing the same register. The RAT now points to the ROB entry until that entry commits ("`wb.py`", "`commit.py`").

Table 3: Requirements of the handout and the behavior of the provided code. All listed requirements are implemented in the submitted simulator.

Feature	Handout requires	Provided code
Issue width	1–4 per cycle, set in the input file	1 only
Input file	Configuration, registers, memory, program	Program only; rest fixed in "`main.py`"
Branch prediction	1-bit predictor, 8-entry BTB, fetch at $n+2$	None; fetch stalls at each branch
Branch types	`Beq` and `Bne`; offset in words	`Bne` only; offset in bytes
Store timing	Write memory only at commit	May write before commit
Register R0	Always 0	Can be written
Output	Status table, registers, non-zero memory	Status table only

Design decisions not specified in the handout.

1. Up to *issue width* instructions commit per cycle.
2. The issue group ends after a branch predicted taken; the target is fetched in the next cycle.
3. Each BTB entry stores the branch PC as a tag. A missing or different entry predicts not taken.
4. When several results wait for the CDB, the result that completed first is broadcast first; ties go to the older instruction.
5. As the handout allows, a load may read memory before the addresses of older stores are known. If one of those stores is later found to have the same address, the load has read a stale value, so the load and all younger instructions are squashed and fetched again, with the same recovery as a branch.
6. An invalid address or branch target is reported only when that instruction commits; wrong-path instructions therefore cannot terminate the simulation.

Changes to the provided inputs. In "`test_case.txt`" the branch offset was changed from -12 to -3: the provided code read the offset in bytes, while the handout defines it in words (its sample uses `Bne R2, R3, -3`). Both select the same target. "`code.in`" was removed because the program and the configuration are now in one input file.

2.3 Timing Rules

Every stage takes at least one cycle, and the simulator applies these rules:

- An instruction issued in cycle c starts EX in cycle $c+1$ at the earliest.
- A value broadcast in cycle c can be used by EX in cycle $c+1$.
- An instruction commits at least one cycle after its broadcast. Branches and stores do not use the CDB; a branch commits after it resolves.
- A ROB entry, reservation station or queue entry freed in a cycle can be reused in the next cycle.

Table 4: Requirements of the handout and the files in "`tomasulo-python/code/`" that implement them.

Feature	Handout requirement	Implemented in
Issue width	1–4 per cycle in order; stop at the first instruction without a free ROB, reservation station or queue entry	"`issue.py`"
Input file	Configuration, `ROB entries`, `Issue width`, registers and memory; case insensitive	"`init.py`"
Branch issue	Branches use the integer ALU reservation stations	"`issue.py`"
Branch prediction	One-bit predictor, 8-entry BTB indexed by PC[2:0]; prediction at issue	"`issue.py`"
Branch resolution	End of EX; target PC+4+offset$\ll$2; `Beq` and `Bne`	"`exe.py`"
Misprediction	Recover the RAT; clear the RS and ROB entries of wrong instructions; fetch at $n+2$	"`exe.py`"
Load/store queue	Program order; dedicated address adder	"`issue.py`", "`exe.py`"
Store forwarding	Checked in MEM, 1 cycle; otherwise read memory, even with unknown store addresses	"`mem.py`"
Load entry	Cleared when the load receives its value	"`mem.py`"
Store timing	Write memory only at commit, then leave the queue	"`commit.py`"
Functional units	Integer adder unpipelined; FP adder and multiplier pipelined; configured count	"`init.py`", "`exe.py`"
CDB	One CDB	"`wb.py`"
Memory	256 bytes, single-ported, non-pipelined	"`mem.py`", "`commit.py`"
Register R0	Hardwired to 0	"`issue.py`", "`commit.py`"
Output	Status table, register values, non-zero memory values	"`print_status.py`"

- A store commits when it is at the ROB head, its address and data are known, and the memory port is free. Its write takes the configured memory cycles; during the write, loads wait for the port.

2.4 Test Cases

All tests are input files in the handout's format: the provided "`tomasulo-python/code/test_case.txt`" and six new tests in "`tomasulo-python/tests/`" (Table 5).

2.5 Results Under Different Issue Widths

"Table 6" gives the cycles (last commit) and IPC of every test at every width. "Table 7" shows the issue and commit cycles of "`t1_independent`", and "Table 8" the status table of the provided test at width 2. The full output of every run is in "`tomasulo-python/results/`".

Independent instructions. "`t1_independent`" decreases from 19 to 14 cycles. At width 4 all twelve instructions issue in three cycles (Table 7). The remaining cycles are set by single-instance resources: the CDB broadcasts one result per cycle (cycles 3 to 10) and the memory port performs the four store writes one after another (cycles 11 to 14). Widths 3 and 4 both take 14 cycles.

Table 5: Test cases and the behavior that each one checks.

Test	Focus	What it checks
"`test_case`" (provided)	Loop	FP dependence chain and a store, followed by a three-iteration loop
"`t1_independent`"	Issue width	Twelve independent integer, FP and store instructions
"`t2_dependencies`"	Hazards	RAW, WAR and WAW hazards inside one issue group; a write to R0
"`t3_loop`"	Branch prediction	Four-iteration loop; first and last `Bne` mispredicted; wrong-path stores must not reach memory
"`t4_branches`"	BTB	Taken and not-taken `Beq`; branches at PC 4 and PC 12 share BTB entry 4
"`t5_memory`"	Memory	Forwarding with store data ready and pending; a load passing an unresolved store; a memory-order violation
"`t6_structural`"	Structural hazards	One reservation station per unit, 4-entry ROB, 1-entry queue; an older writer of F5 completing after a younger one

Table 6: Cycles and IPC per test and issue width W. Mispredictions and memory-order violations ("viol.") are counted when detected; "squashed" counts squashed instructions.

	Cycles				IPC			
Test (instructions)	W=1	2	3	4	W=1	W=4	Mispred.	Squashed (W=1/4)
"`test_case`" (15)	41	35	33	32	0.366	0.469	2	2 / 4
"`t1_independent`" (12)	19	15	14	14	0.632	0.857	0	0 / 0
"`t2_dependencies`" (12)	31	28	28	28	0.387	0.429	0	0 / 0
"`t3_loop`" (26)	38	32	30	30	0.684	0.867	2	3 / 7
"`t4_branches`" (31)	44	38	37	36	0.705	0.861	3	3 / 5
"`t5_memory`" (13)	25	21	21	21	0.520	0.619	0	3 / 0
"`t6_structural`" (7)	21	20	20	20	0.333	0.350	0	0 / 0

Dependences and structural limits. "`t2_dependencies`" decreases from 31 to 28 cycles at width 2 and does not improve further, because its instructions form dependence chains. "`t6_structural`" improves by one cycle: its 4-entry ROB fills behind the 10-cycle multiply, so issue stalls at every width. The same test also verifies the renaming correction: `Add.d F6, F5, F5` issues after the multiply has broadcast and still reads F5 from the younger `Add.d` (F6 = 14.0).

Branches. The number of mispredictions is the same at every width (2, 2 and 3 in "`test_case`", "`t3_loop`" and "`t4_branches`"), because the predictor receives the same branch outcomes. In "`test_case`" and "`t3_loop`" the first loop branch is mispredicted because the BTB is still empty, and the loop exit is mispredicted because the one-bit predictor stores the previous outcome (taken). In "`t4_branches`" the exit test at PC 4 overwrites the shared BTB entry in every iteration, so both taken executions of the loop branch at PC 12 are mispredicted; the third misprediction is the `Beq` at PC 1 with an empty BTB. Each misprediction adds the recovery cycle: in "Table 8" the first `Bne` resolves in cycle 6 and the correct path issues in cycle 8 ($n + 2$). Wider issue fetches more wrong-path instructions before a branch resolves ("`t3_loop`": 3 squashed at widths 1 and 2, 7 at widths 3 and 4), but the total cycle count still decreases.

Table 7: Issue and commit cycles of "`t1_independent`" for each issue width.

	ISSUE				COMMIT			
Instruction	W=1	2	3	4	W=1	2	3	4
`Addi R1, R0, 1`	1	1	1	1	4	4	4	4
`Addi R2, R0, 2`	2	1	1	1	5	5	5	5
`Add.d F1, F10, F11`	3	2	1	1	8	7	6	6
`Mult.d F2, F12, F13`	4	2	2	1	11	10	10	10
`Addi R3, R0, 3`	5	3	2	2	12	10	10	10
`Sub.d F3, F10, F11`	6	3	2	2	13	11	10	10
`Mult.d F4, F10, F12`	7	4	3	2	14	11	11	11
`Addi R4, R0, 4`	8	4	3	2	15	12	11	11
`Sd F10, 0(R0)`	9	5	3	3	16	12	11	11
`Sd F11, 4(R0)`	10	5	4	3	17	13	12	12
`Sd F12, 8(R0)`	11	6	4	3	18	14	13	13
`Sd F13, 12(R0)`	12	6	4	3	19	15	14	14

Memory. In "`t5_memory`" at width 1, `Ld F5, 12(R0)` reads memory in the cycle in which the aliasing `Sd F3, 0(R1)` computes its address. The load and the two instructions after it are squashed and fetched again, and the load then reads the stored value. At widths 2 to 4 the memory port is busy with store writes until that store has committed, so the load reads the new value and no violation occurs. The final registers and memory are the same at every width.

2.6 Validation

"`tests/run_tests.py`" runs every test at widths 1 to 4 and checks each run:

- **Results:** the committed instructions, final registers and memory equal an in-order execution of the same program.
- **Timing**, for every committed instruction: issue and commit in program order, at most W per cycle; at most one CDB broadcast per cycle; EX takes the configured cycles; MEM, WB and COMMIT follow in order; stores write memory at commit; memory accesses never overlap; the correct path is fetched no earlier than $n + 2$ after a misprediction.
- **BTB:** a short loop with branches at PC 1 and PC 5, which use separate entries only in an 8-entry BTB indexed by PC[2:0], must give 3 mispredictions at every width: the first `Beq` and the first `Bne` with an empty BTB, and the loop exit.

All 28 runs and the 4 BTB checks pass with Python 3.11 on Windows and Python 3.12 on Linux. The width-1 run of the provided test was also checked cycle by cycle by hand.

To confirm that these checks detect errors, twelve single errors were inserted one at a time into copies of the simulator, for example two CDB broadcasts per cycle, refetch at $n+1$, a 4-entry BTB and no store forwarding (this test is not part of the submission). The script reported a failure for eleven of them. The twelfth, allowing R0 to be renamed, cannot change any result because every read of R0 returns 0. The BTB check also exposed a crash when a mispredicted branch is the last instruction of a program; it was corrected in "`exe.py`".

Table 8: Instruction status table of the provided "`test_case`" at issue width 2 (committed instructions; EX and MEM give the first and last cycle).

Instruction	ISSUE	EX	MEM	WB	COMMIT
`Ld F2, 0(R1)`	1	2–2	3–7	8	9
`Mult.d F4, F2, F20`	1	9–23	–	24	25
`Ld F6, 0(R2)`	2	3–3	8–12	13	25
`Add.d F6, F4, F6`	2	25–28	–	29	30
`Sd F6, 0(R2)`	3	4–4	30–34	–	30
`Addi R1, R1, -4`	3	4–4	–	5	31
`Addi R2, R2, -4`	4	5–5	–	6	31
`Bne R1, R0, -3`	4	6–6	–	–	32
`Addi R1, R1, -4`	8	9–9	–	10	32
`Addi R2, R2, -4`	8	10–10	–	11	33
`Bne R1, R0, -3`	9	11–11	–	–	33
`Addi R1, R1, -4`	10	12–12	–	14	34
`Addi R2, R2, -4`	10	13–13	–	15	34
`Bne R1, R0, -3`	11	15–15	–	–	35
`Add.d F20, F2, F2`	17	18–21	–	22	35

2.7 What Works and What Does Not

All requirements in "Table 4" are met in these tests, and no errors are known. Remaining simplifications:

- FP values are Python double-precision numbers, not single precision.
- Only word-aligned addresses inside the 256-byte memory are accepted.
- A memory-order violation squashes everything from the load on; there is no selective re-execution.
- The BTB is written when a branch resolves, so a wrong-path branch can update it before it is squashed.
- The status table lists committed instructions only; squashed ones are counted in the summary line.
- The handout's sample program is not used as a test: it does not terminate (`Bne R2, R3, -3` with R2 = 20 and R3 = 0), and `Ld F4, 8(R1)` with R1 = 10 reads address 18, which is not word aligned.

2.8 Files

Paths are relative to "`o3-simulator/tomasulo-python/`". "`code/`" holds the simulator ("`main.py`" and one module per stage) and the provided "`test_case.txt`"; "`tests/`" holds the test inputs and "`run_tests.py`"; "`results/`" holds the output of "`main.py`" for every test and width, and "`summary.txt`". "`o3-simulator/README.md`" gives the commands.

3 Conclusion

Part 1. The Pentium M-style predictor, a 4096-entry bimodal table with a 4-way, 512-set tagged global table, averages 5.849 MPKI on the 20 traces. It mispredicts less than the bimodal table alone (10.267 MPKI) and the supplied gshare predictor (6.306 MPKI), with less than half of gshare's storage. Among the PM configurations, the largest improvement comes from creating global entries only after a bimodal misprediction (7.758 to 5.935 MPKI).

Part 2. The Tomasulo simulator now issues 1 to 4 instructions per cycle in order and follows the processor described in the handout, including branch prediction with recovery and the load/store rules. All seven tests give correct results at every issue width. The largest relative reduction from wider issue is for independent instructions ("`t1_independent`": 19 to 14 cycles, 26%); dependences, the single CDB and the single memory port limit further gains.

References

[1] V. Uzelac and A. Milenkovic, "Experiment Flows and Microbenchmarks for Reverse Engineering of Branch Predictor Structures," in *Proc. IEEE Int. Symp. on Performance Analysis of Systems and Software (ISPASS)*, 2009. `https://alexmilenkovich.github.io/publications/files/vuam_ispass09.pdf`.

[2] 2nd JILP Championship Branch Prediction Competition (CBP-2) simulation infrastructure, 2006. It includes the reference `gshare_predictor`.